MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK

MODUL 6
Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP

2026

MODUL PRAKTIKUM P6
Polymorphism, Method Overriding, Method Overloading, dan Dynamic Binding

1. Tujuan Praktikum

Setelah menyelesaikan praktikum, mahasiswa mampu:

- ✓ Menjelaskan konsep **polymorphism**.
- ✓ Membedakan **method overloading** dan **method overriding**.
- ✓ Menerapkan overriding pada subclass.
- ✓ Menggunakan reference superclass untuk object subclass.
- ✓ Menjelaskan proses **dynamic binding**.
- ✓ Menerapkan polymorphic behavior pada proyek.
- ✓ Melakukan refactoring project hasil P5.
- ✓ Menghasilkan *project increment* melalui Agile Sprint.

2. Review P5

Pada P5 mahasiswa telah membuat:

- superclass,
- subclass,
- inheritance,
- `extends`,
- `super`,
- constructor chaining.

Contoh:

```
class Kendaraan {
}

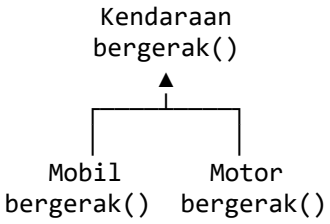
class Mobil extends Kendaraan {
}

class Motor extends Kendaraan {
}
```

Pada P6, subclass tidak hanya mewarisi struktur superclass.

Subclass dapat memiliki **perilaku berbeda untuk method yang sama**.

Contoh:



Konsep tersebut disebut **polymorphism**.

3. Konsep Singkat

A. Polymorphism

Polymorphism berarti:

> Satu bentuk pemanggilan dapat menghasilkan perilaku yang berbeda sesuai object yang digunakan.

Contoh:

```
Kendaraan kendaraan1 = new Mobil();
Kendaraan kendaraan2 = new Motor();
```

Walaupun reference bertipe:

```
Kendaraan
```

object sebenarnya dapat berupa:

```
Mobil
Motor
```

4. Method Overriding

Overriding terjadi ketika subclass mendefinisikan ulang method dari superclass.

Superclass:

```
class Kendaraan {

    public void bergerak() {
        System.out.println(
            "Kendaraan bergerak"
        );
    }
}
```

Subclass:

```
class Mobil extends Kendaraan {

    @Override
    public void bergerak() {
        System.out.println(
            "Mobil berjalan menggunakan roda"
        );
    }
}
```

Subclass lain:

```
class Kapal extends Kendaraan {

    @Override
    public void bergerak() {
        System.out.println(
            "Kapal bergerak di air"
        );
    }
}
```

5. Dynamic Binding

Perhatikan:

```
Kendaraan kendaraan =
    new Mobil();

kendaraan.bergerak();
```

Output:

```
Mobil berjalan menggunakan roda
```

Method yang dijalankan ditentukan berdasarkan **object aktual**, bukan hanya tipe reference.

Proses tersebut disebut:

```
Dynamic Binding
```

atau:

```
Dynamic Method Dispatch
```

6. Upcasting

Contoh:

```
Mobil mobil =
    new Mobil();

Kendaraan kendaraan =
    mobil;
```

Object `Mobil` diperlakukan sebagai `Kendaraan`.

Bentuk ringkas:

```
Kendaraan kendaraan =
    new Mobil();
```

Hal ini memungkinkan penggunaan polymorphism.

7. Method Overloading

Overloading berbeda dengan overriding.

Overloading terjadi ketika terdapat beberapa method dengan:

```
Nama sama
Parameter berbeda
```

Contoh:

```
class Pembayaran {

    public void bayar(
        double total
    ) {
        System.out.println(total);
    }

    public void bayar(
        double total,
        double diskon
    ) {
        System.out.println(
            total - diskon
        );
    }
}
```

8. Overloading vs Overriding

Aspek	Overloading	Overriding
Nama method	Sama	Sama
Parameter	Berbeda	Sama
Class	Umumnya satu class	Superclass – subclass
Tujuan	Variasi pemanggilan	Perbedaan behavior
Polymorphism	Compile-time	Runtime

Fokus utama P6 adalah:

> Runtime Polymorphism melalui Method Overriding.

9. Contoh Polymorphism

Superclass:

```
class Pegawai {

    public void hitungGaji() {
        System.out.println(
            "Menghitung gaji pegawai"
        );
    }
}
```

Subclass:

```
class PegawaiTetap
        extends Pegawai {

    @Override
    public void hitungGaji() {
        System.out.println(
            "Gaji tetap + tunjangan"
        );
    }
}
class PegawaiHarian
        extends Pegawai {

    @Override
    public void hitungGaji() {
```

```
            System.out.println(
                "Jumlah hari x upah harian"
            );
        }
    }
```

Pengujian:

```
    Pegawai p1 =
        new PegawaiTetap();

    Pegawai p2 =
        new PegawaiHarian();

    p1.hitungGaji();
    p2.hitungGaji();
```

Output berbeda walaupun reference sama-sama bertipe `Pegawai`.

10. Polymorphic Collection

Polymorphism menjadi lebih berguna ketika beberapa object disimpan dalam collection atau array yang sama.

Contoh:

```
Kendaraan[] daftar = {
    new Mobil(),
    new Motor(),
    new Kapal()
};
```

Kemudian:

```
for (
    Kendaraan kendaraan :
    daftar
) {

    kendaraan.bergerak();
}
```

Setiap object menjalankan method sesuai implementasinya sendiri.

11. Agile Sprint – P6

Sprint Goal

Mengembangkan behavior object pada proyek melalui method overriding dan polymorphism sehingga subclass dapat merespons pemanggilan method yang sama dengan perilaku berbeda.

Sprint Backlog

Contoh:

ID	Sprint Backlog	Status
SB-01	Audit superclass dan subclass P5	To Do
SB-02	Menentukan behavior yang dapat dioverride	To Do
SB-03	Implementasi overriding	To Do
SB-04	Implementasi overloading sederhana	To Do
SB-05	Implementasi upcasting	To Do
SB-06	Membuat polymorphic collection	To Do
SB-07	Melakukan pengujian dynamic binding	To Do

Status:



12. Tugas Praktikum

Tugas P6 – Implementasi Polymorphism pada Project

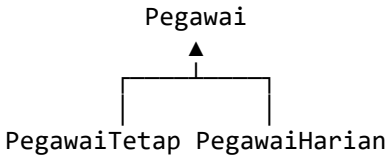
Gunakan **project hasil P5**.

Mahasiswa tidak membuat project baru.

13. Bagian A – Audit Behavior

Pilih hierarchy hasil P5.

Contoh:



Cari minimal **1 behavior** yang:

- dimiliki superclass, tetapi memiliki implementasi berbeda pada subclass.

Contoh:

Superclass	Subclass	Method
Pegawai	PegawaiTetap	hitungGaji()
Pegawai	PegawaiHarian	hitungGaji()

14. Bagian B – Method Overriding

Superclass:

```
public class Pegawai {

    public void tampilkanPeran() {

        System.out.println(
            "Pegawai"
        );
    }
}
```

Subclass:

```
public class Manager
        extends Pegawai {

    @Override
    public void tampilkanPeran() {

        System.out.println(
            "Manager"
        );
    }
}
```

Buat minimal **2 subclass** yang melakukan overriding.

15. Bagian C – Method Overloading

Tambahkan minimal **1 contoh overloading** yang relevan.

Contoh:

```
public void cari(
    String nama
) {
    // ...
}
public void cari(
    int id
) {
    // ...
}
```

Overloading harus memiliki alasan penggunaan yang jelas.

16. Bagian D – Upcasting

Buat reference superclass untuk object subclass.

Contoh:

```
Pegawai pegawai1 =
    new Manager();

Pegawai pegawai2 =
    new Staff();
```

Tidak cukup hanya membuat:

```
Manager manager =
    new Manager();
```

Mahasiswa harus menunjukkan polymorphic reference.

17. Bagian E – Polymorphic Collection

Gunakan minimal **3 object subclass**.

Contoh:

```
Pegawai[] daftarPegawai = {

    new Manager(),
    new Staff(),
    new Supervisor()
};
```

Lakukan iterasi:

```
for (
    Pegawai pegawai :
    daftarPegawai
) {

    pegawai.tampilkanPeran();
}
```

18. Bagian F – Pengujian Dynamic Binding

Buat tabel pengujian:

Reference	Object Aktual	Method	Output
Pegawai	Manager	tampilkanPeran()	Manager
Pegawai	Staff	tampilkanPeran()	Staff

Mahasiswa harus dapat menjelaskan:

> Mengapa method yang dijalankan berbeda walaupun reference memiliki tipe yang sama?

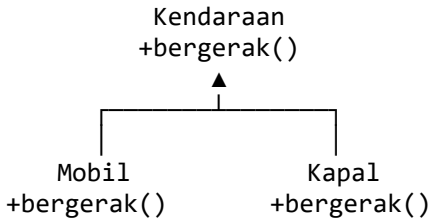
19. Update Class Diagram

Perbarui class diagram proyek.

Diagram harus menampilkan:

- superclass,
- subclass,
- generalization,
- method yang dioverride,
- relasi P4 yang masih digunakan.

Contoh:



20. Luaran Tugas

Kumpulkan:

```
P6_NRP_Nama/
│
├─── README.pdf
│
└─── src/
     ├─── Main.java
     ├─── Superclass.java
     ├─── Subclass1.java
     ├─── Subclass2.java
     └─── ClassLain.java
```

`README.pdf` maksimal **3 halaman**.

Halaman 1

Nama project

Sprint Goal

Sprint Backlog

Hierarchy class P5

Behavior yang dipilih untuk polymorphism

Halaman 2

Class Diagram terbaru

Contoh overriding

Contoh overloading

Penjelasan upcasting

Halaman 3

Screenshot hasil running

Tabel pengujian dynamic binding

Sprint Review

Sprint Retrospective

21. Sprint Review

Item	Hasil
Superclass dan subclass tersedia	...
Method overriding berhasil	...
Minimal 2 subclass memiliki behavior berbeda	...
Method overloading tersedia	...
Upcasting berhasil	...
Polymorphic collection berhasil	...
Dynamic binding dapat dibuktikan	...
Program berjalan	...
Kendala	...

22. Sprint Retrospective

Jawab singkat.

What Went Well?

Behavior polymorphic apa yang berhasil diterapkan?

What Went Wrong?

Apa kesulitan dalam membedakan overriding dan overloading?

Improvement

Apa yang perlu diperbaiki pada hierarchy atau behavior object?

Maksimal **3–5 kalimat**.

23. Definition of Done

P6 selesai apabila:

- ✓ Menggunakan project P5.
- ✓ Minimal 1 superclass digunakan.
- ✓ Minimal 2 subclass digunakan.
- ✓ Terdapat method pada superclass yang dioverride.
- ✓ Minimal 2 subclass melakukan overriding.
- ✓ Menggunakan annotation @Override.
- ✓ Terdapat minimal 1 contoh overloading.
- ✓ Terdapat penggunaan upcasting.
- ✓ Terdapat polymorphic reference.
- ✓ Minimal 3 object digunakan dalam pengujian polymorphism.
- ✓ Dynamic binding berhasil ditunjukkan.
- ✓ Class Diagram diperbarui.
- ✓ Program berhasil dikompilasi.
- ✓ Program berhasil dijalankan.
- ✓ Sprint Backlog diperbarui.
- ✓ Sprint Review tersedia.
- ✓ Sprint Retrospective tersedia.

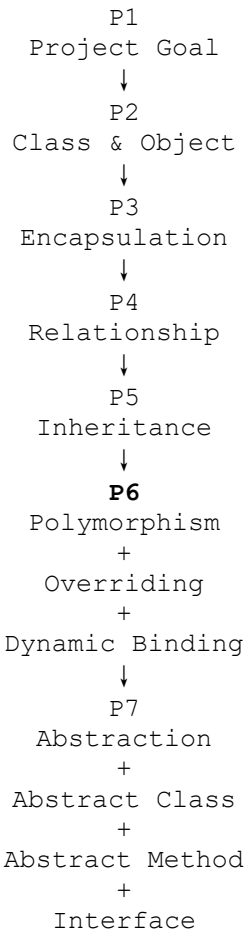
24. Penilaian

Komponen	Bobot
Identifikasi behavior polymorphic	10%
Method overriding	25%
Method overloading	10%
Upcasting & polymorphic reference	20%
Dynamic binding & pengujian	20%

Komponen	Bobot
Class Diagram	5%
Agile Sprint & dokumentasi	10%
Total	**100%**

25. Target Sprint Berikutnya

Hasil P6 menjadi dasar untuk **P7 – Abstract Class dan Interface**.



Pada P7, mahasiswa akan mengevaluasi hierarchy yang dibuat pada P5–P6 dan menentukan class yang seharusnya **tidak dibuat menjadi object secara langsung**, kemudian menerapkan **abstract class dan interface**.